```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

```
data = pd.read_csv('./diabetes.csv')
```

```
print(data.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   Pedigree  Age  Outcome
0     0.627   50        1
1     0.351   31        0
2     0.672   32        1
3     0.167   21        0
4     2.288   33        1
```

```
data.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
Pedigree	0
Age	0
Outcome	0

dtype: int64

```
cols_to_replace = ['Glucose','BloodPressure','SkinThickness','Insulin','BMI']
for column in cols_to_replace:
    data[column].replace(0, np.nan, inplace=True)
    data[column].fillna(round(data[column].mean(skipna=True)), inplace=True)
```

```
/tmp/ipython-input-3377949948.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chair
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = c


  data[column].replace(0, np.nan, inplace=True)
/tmp/ipython-input-3377949948.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chair
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = c


  data[column].fillna(round(data[column].mean(skipna=True)), inplace=True)
```

```
X = data.iloc[:, :8]   # first 8 columns are features
Y = data['Outcome']
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)
```

```
# Initialize KNN
knn = KNeighborsClassifier(n_neighbors=5)  # you can change k
knn.fit(X_train, Y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
knn_pred = knn.predict(X_test)
```

```
# Metrics
cm = confusion_matrix(Y_test, knn_pred)
accuracy = accuracy_score(Y_test, knn_pred)
error_rate = 1 - accuracy
precision = precision_score(Y_test, knn_pred)
recall = recall_score(Y_test, knn_pred)
f1 = f1_score(Y_test, knn_pred)
```

```
# Print results
print("Confusion Matrix:\n", cm)
print("Accuracy Score:", accuracy)
print("Error Rate:", error_rate)
print("Precision Score:", precision)
print("Recall Score:", recall)
print("F1 Score:", f1)
```

```
Confusion Matrix:
 [[88 19]
 [19 28]]
Accuracy Score: 0.7532467532467533
Error Rate: 0.24675324675324672
Precision Score: 0.5957446808510638
Recall Score: 0.5957446808510638
F1 Score: 0.5957446808510638
```